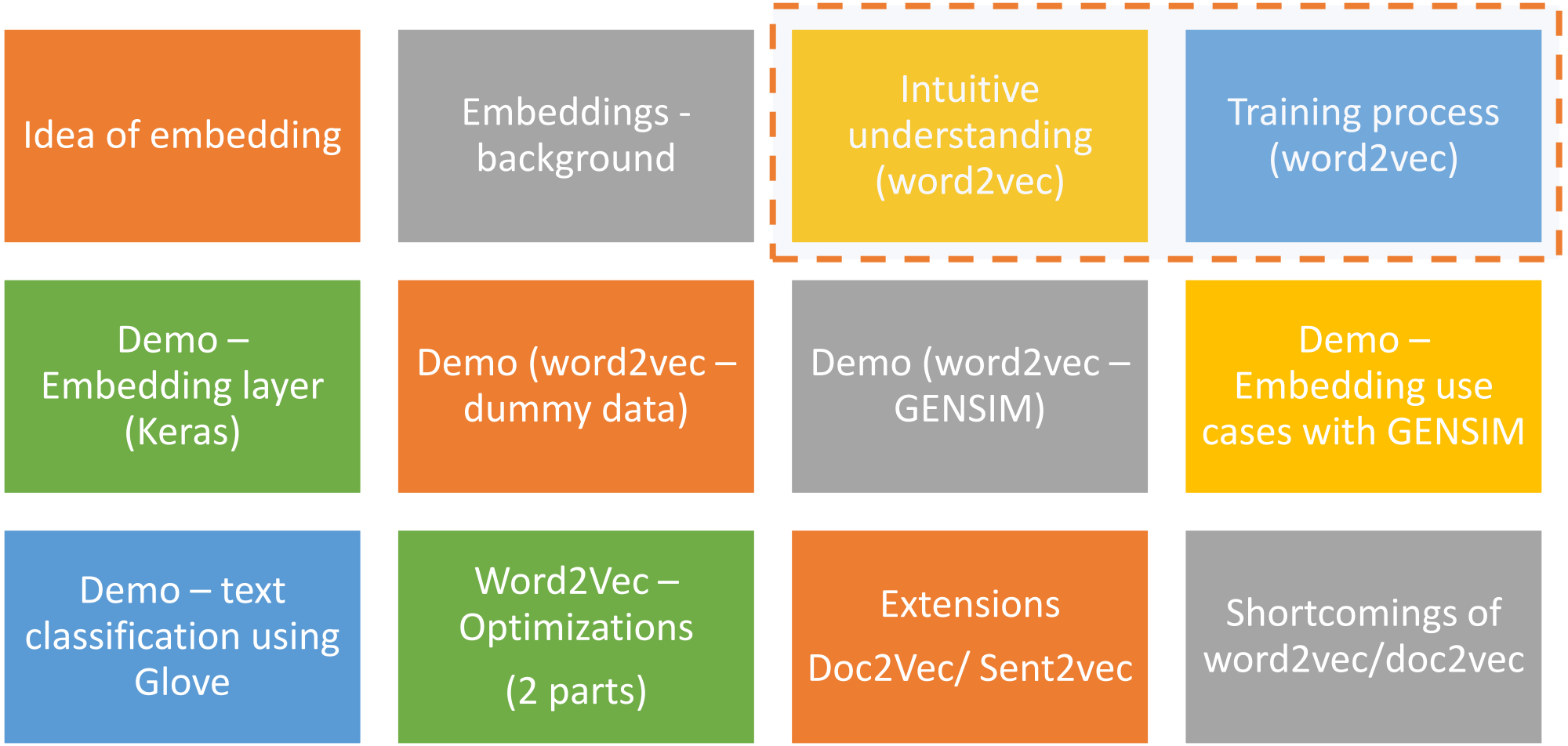
Embeddings (word2vec)

"Visualizing Language and Images: The Role of Embeddings"

Pre-reqs

- Python/ Data Science
- Machine learning concepts
- Deep learning architectures (ANN, MLPs)
- NLP – text processing (GENSIM, SPACY, NLTK)

Topics to cover



Word2vec – basic idea

premise - a word's meaning is intricately linked to its surrounding words

context is defined by neighboring words.

context is conceptualized as N words preceding and succeeding the current word, with N being a hyperparameter.

A larger N can lead to the creation of more nuanced embeddings

In the original paper, N is set at 4–5

Word embeddings - specifications

- 8 dimensions
- 32 dimensions
- 50 dimensions
- 100 dimensions
- 300 dimensions
- 600 dimensions
- 1024 dimensions (typically for large datasets)
- 4K dimensions.
- ….

Ways to build word embeddings

training your own word embeddings

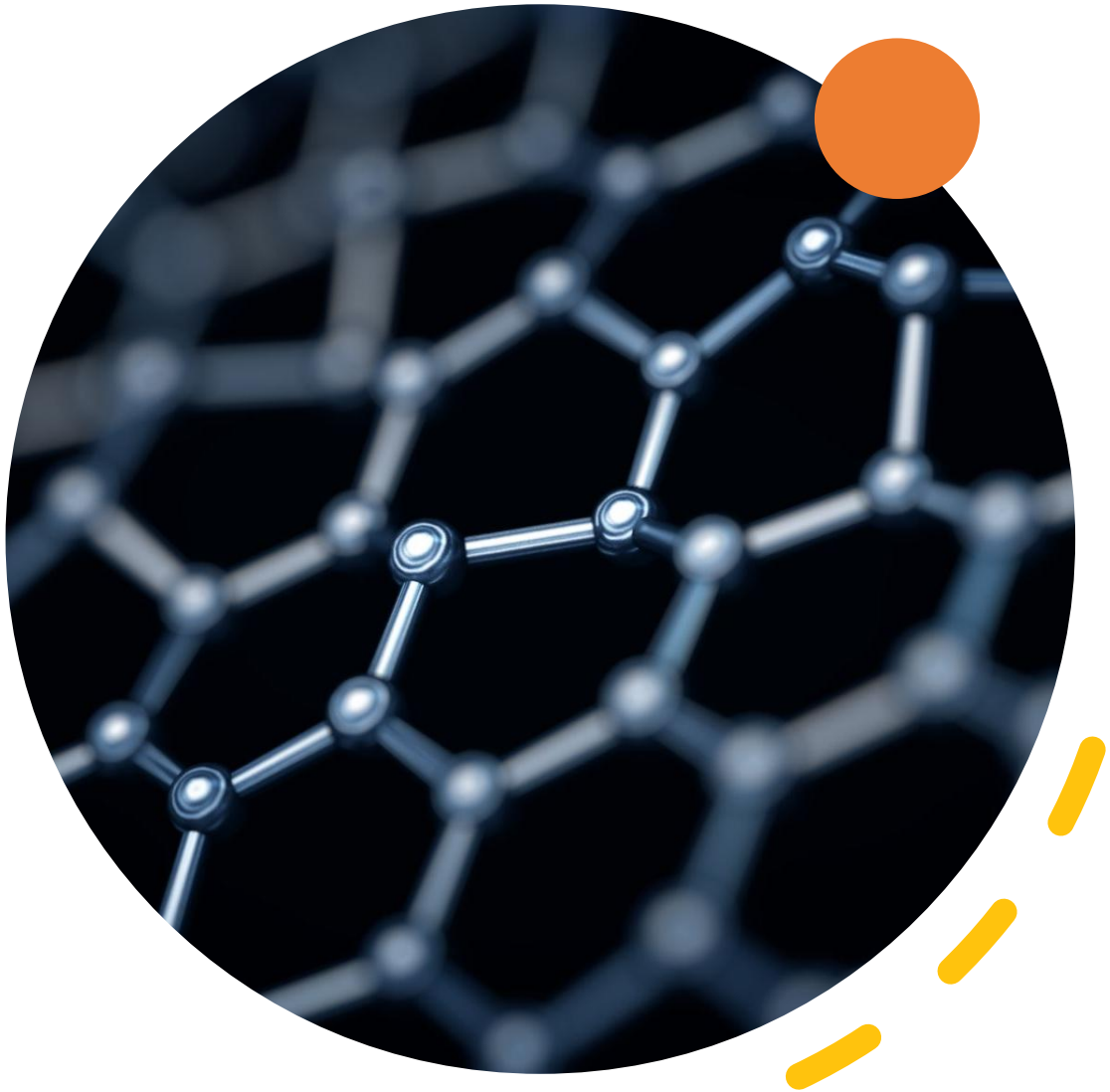
Using large amounts of unannotated plain text, learns relationships between words.

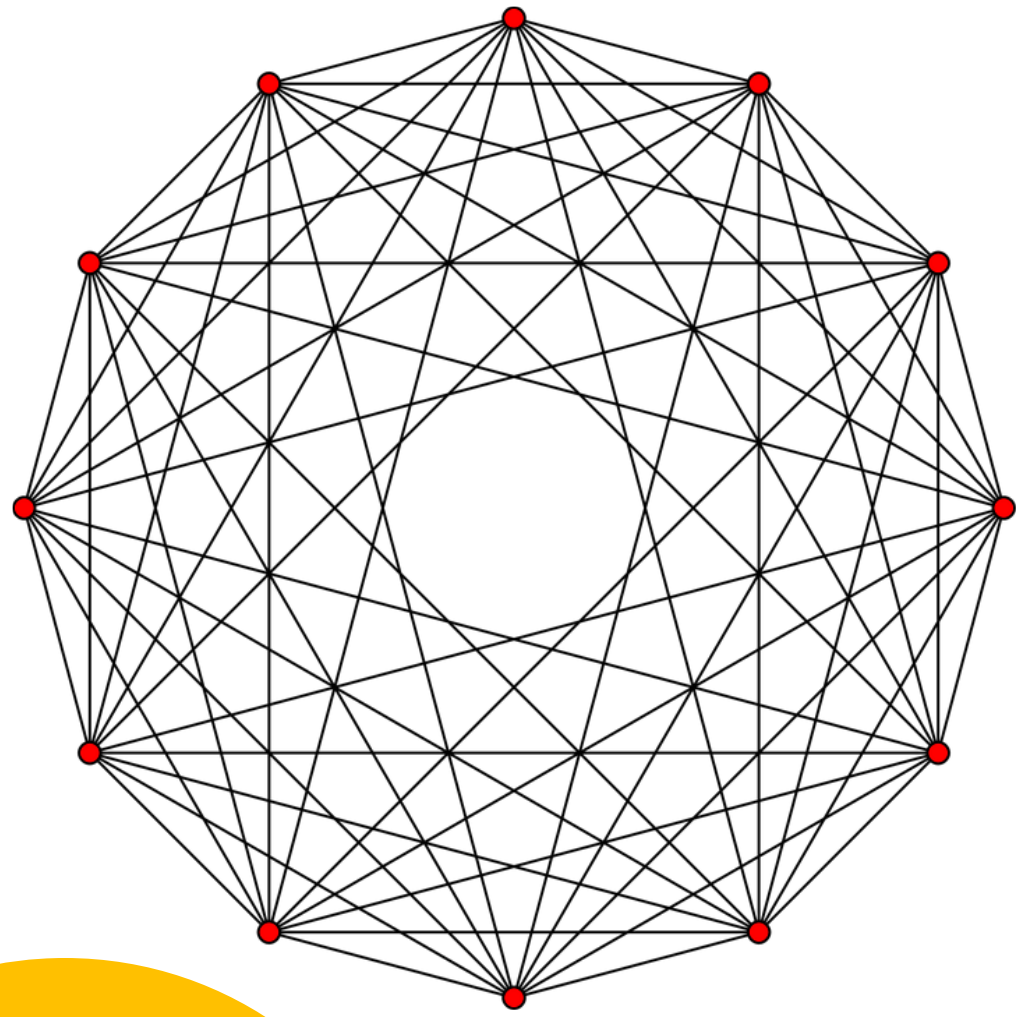
using a 3 layer shallow NN

pre-trained embeddings available such as GloVe, fastText and ELMo

Also called transfer learning

extensions of Word2Vec such as Doc2Vec and the most recent Code2Vec where documents and codes are turned into vectors.

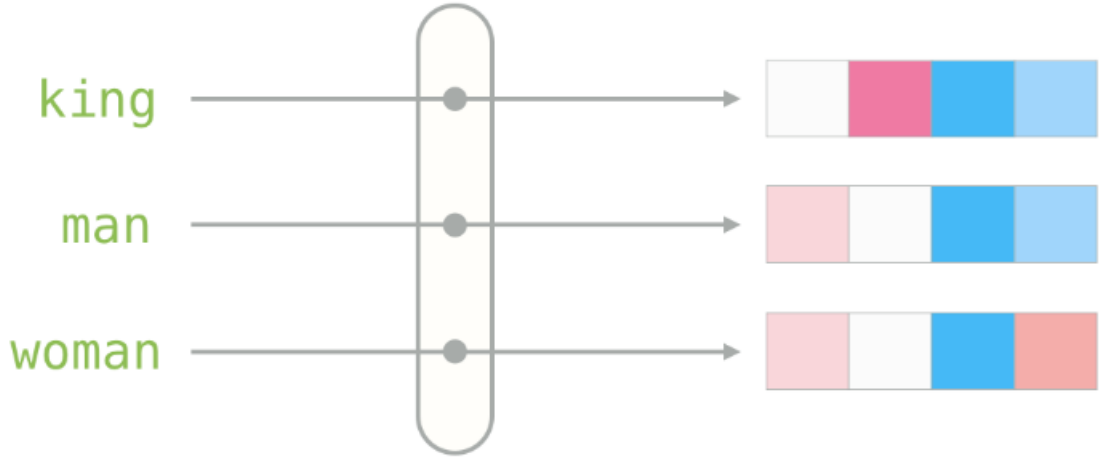




Word2vec

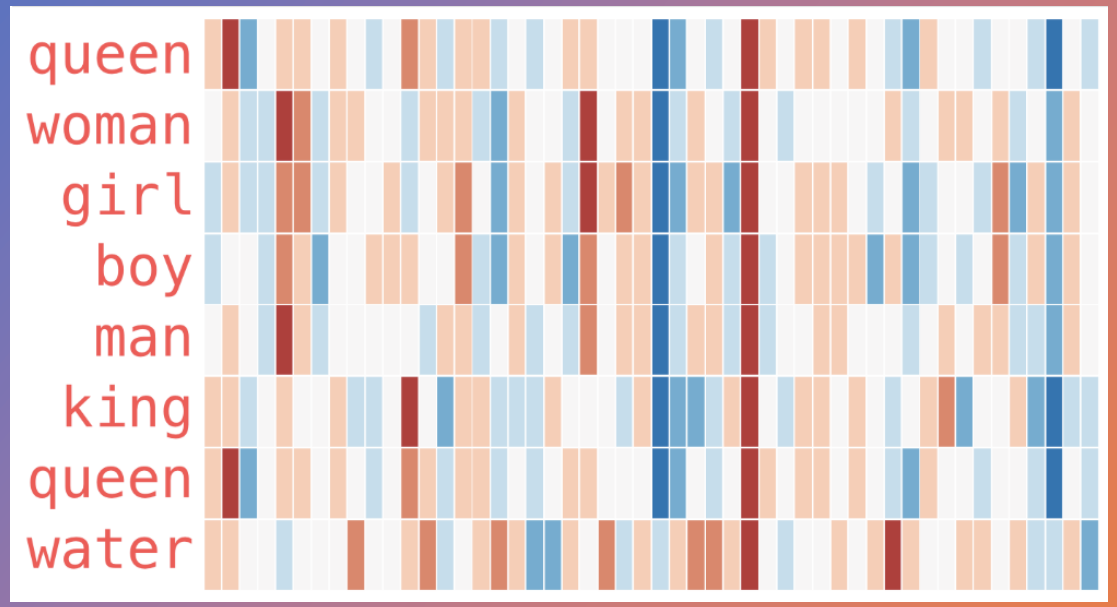
- is a group of related models (extensions etc)
- models are shallow, 3-layer neural networks
  - input layer
  - hidden layer
  - output layer
- takes as its input a large corpus of text and produces a vector space,
- typically, of several hundred dimensions, with each unique word in the corpus being assigned a corresponding entry in the vector space.
- Word vectors are positioned in the vector space such that words that share common contexts in the corpus are located close to one another in the space

Example - how the embeddings look like



[0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 , -0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 , -0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 , -1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042]

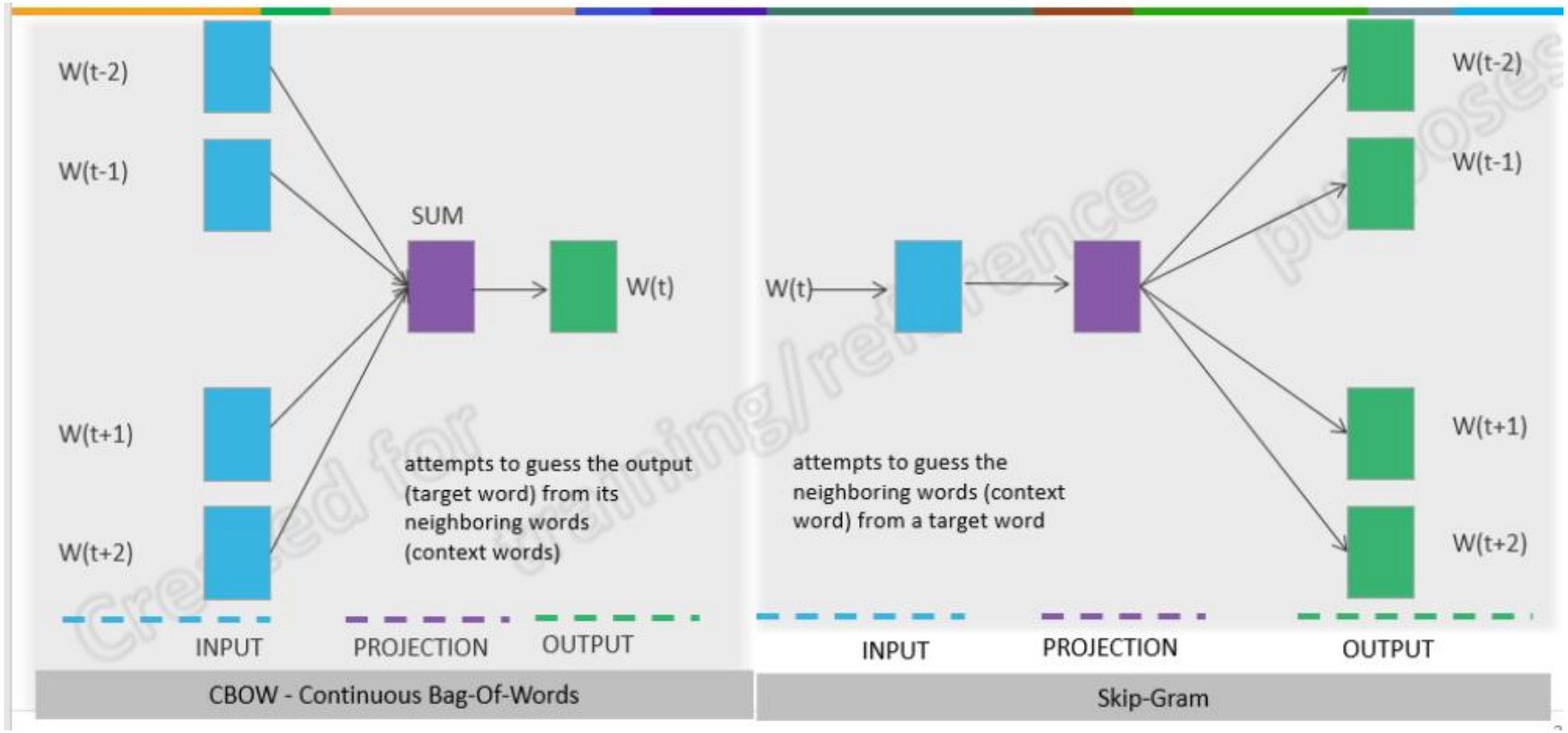
word embedding for the word “king”



Similar vectors

- a straight red column through all of these different words.
- “woman” and “girl” are similar to each other in a lot of places.
- same with “man” and “boy”
- All but the last word … for example, see that blue column going all the way down and stopping before the embedding for “water”.

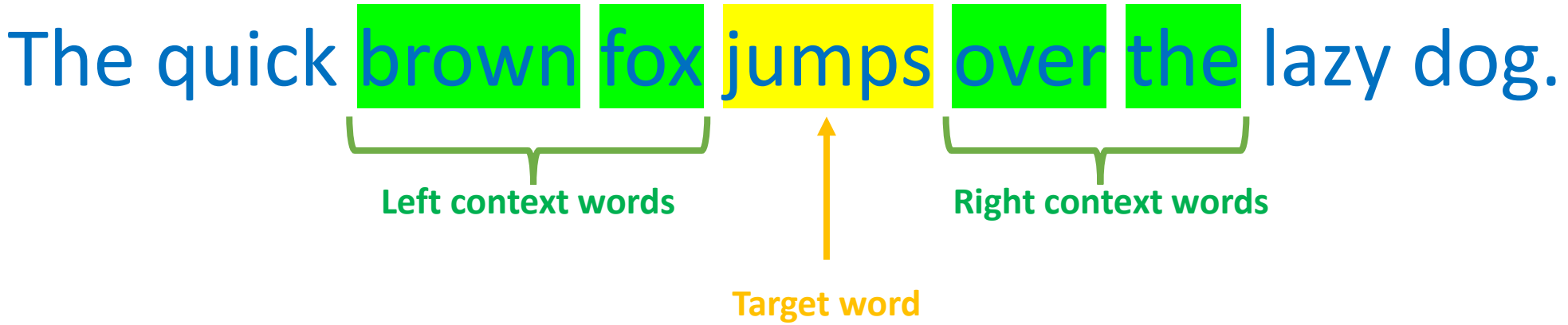
Two forms of NN for word2vec



anticipate a given word by considering its surrounding context words.

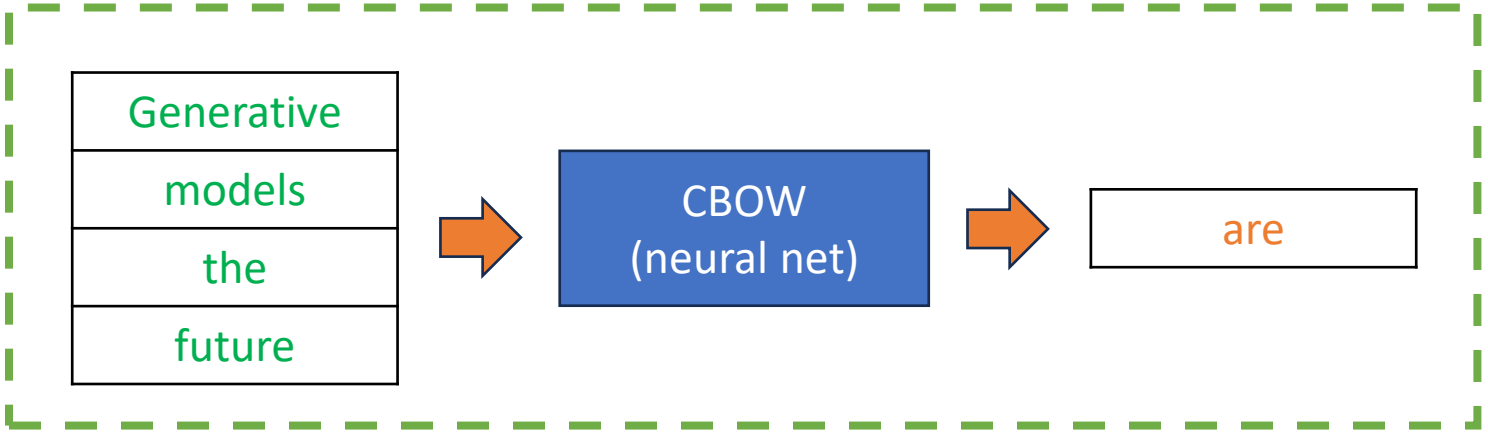
forecast context words by analyzing the present word.

Example (N=2)



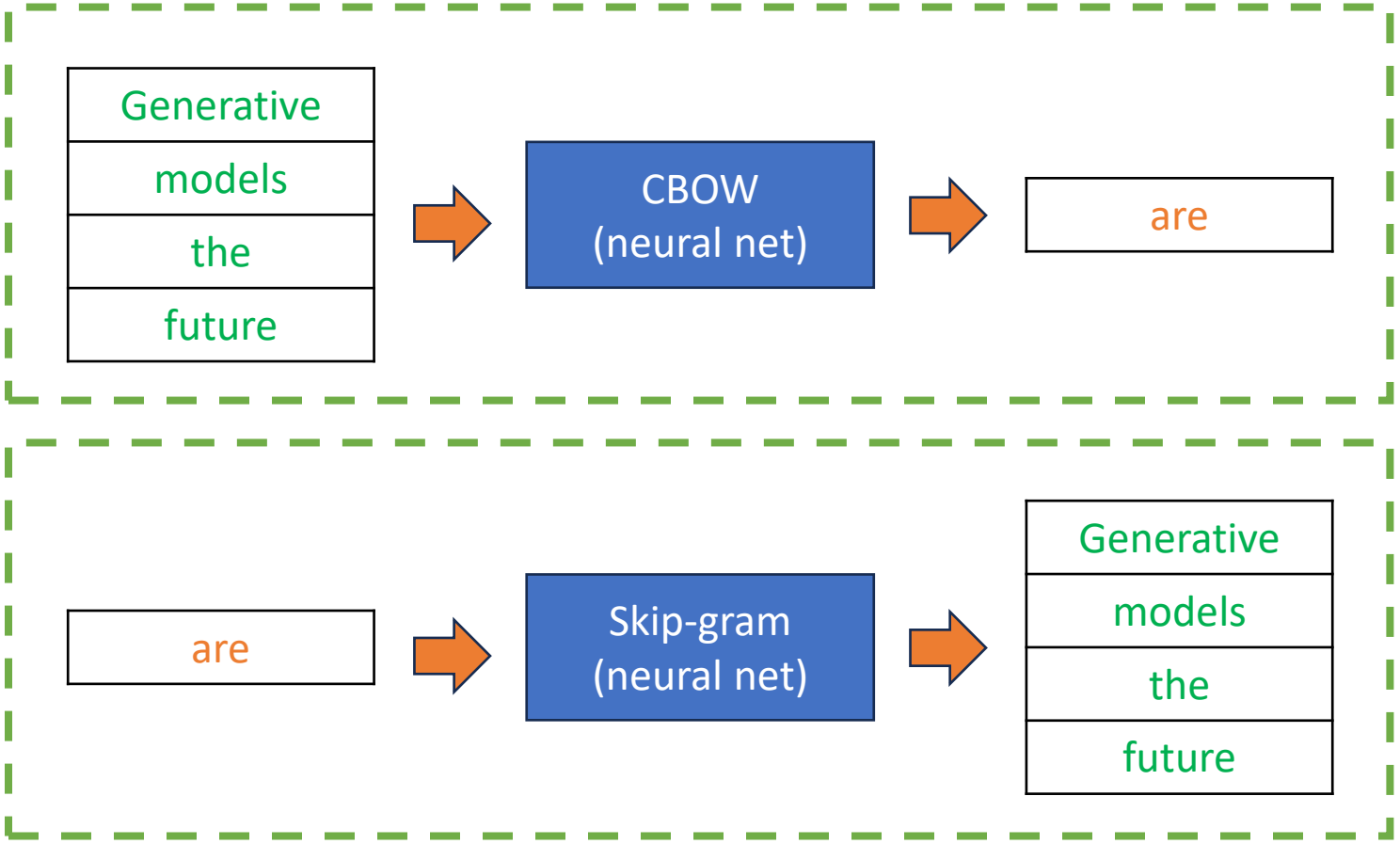
Example (data for the model)

sentence : "Generative models are the future of AI"



Example (data for the model)

sentence : "Generative models are the future of AI"



Training process

Neural Net – models for word2vec

The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog
The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog
The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog
The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog

$y_{c==1}$ $y_{c==2}$ x_k $y_{c==3}$ $y_{c==4}$

The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog
The	Quick	Brown	Fox	Jumps	Over	The	Lazy	dog
The	Quick		Fox	Jumps	Over	The	Lazy	Dog

data preparation (sliding window, N=2)

- Suppose we have a sliding window of a fixed size moving along a sentence:
  - the word in the middle is the “target” and
  - those on its left and right within the sliding window are the context words.

Data samples for the NN of word2vec

- Given the sentence: "He says make India great again."
- Windows:
  - For each center word, the window would consist of 3 words to the left and 3 words to the right
    - (when available):

1. [-, -, -, **He**, says, make, India]
2. [-, -, He, **says**, make, India, great]
3. [-, He, says, **make**, India, great, again]
4. [He, says, make, **India**, great, again, -]
5. [says, make, India, **great**, again, -, -]
6. [make, India, great, **again**, -, -, -]

Let's generate training samples using the first window

Given the window: [-, -, -, **He**, says, make, India]

- Skip-Gram Training Samples:
- For the center word "He", the surrounding words in its window are used as context:

Input (center word)	Output (Context)
He	says
He	make
He	India

- CBOW Training Samples:
- Using the surrounding words in the window, we try to predict the center word "He":

Input (Context words)	Output (Center word)
says, make, India	He

Training samples

Skip-Gram Training Samples:

for each center word, we use it as the input to predict its context (surrounding words).

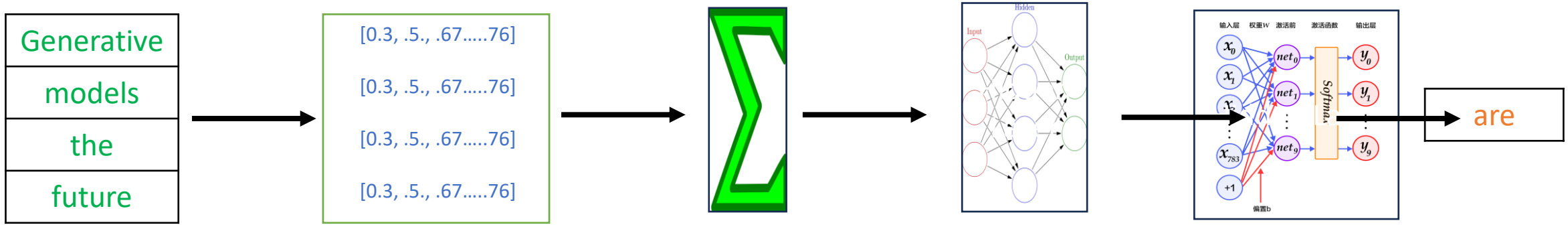
Input (Center Word)	Output (Context Word)
He	says
He	make
He	India

CBOW Training Samples:

In CBOW, we use the surrounding words as input to predict the center word.

Input (Context Words)	Output (Center Word)
says	He
make	He
India	He

Architecture



Context words

CBOW uses a fixed-size context window (N words before and N words after the target word).

Embedding layer

The context words are converted into their respective embeddings.

Averaging

embeddings of context words are aggregated (typically averaged) to form a single vector representation.

Shallow NN

hidden layer with nonlinear activation functions

loss functions include categorical cross-entropy

Softmax

The hidden layer's output is connected to the output layer, which produces a probability distribution over the vocabulary.

Softmax activation function is applied to convert the output into probabilities.

Target word

Input

- Given our window from the previous example:
  - [-, -, -, He, says, make, India],
- our input context words for CBOW would be: says, make, and India.

Embedding layer

serves as a lookup table, associating integer indices with dense vectors that represent the embeddings of specific words.

Like a parameterized table where each word is assigned a unique dense vector.

dimensionality embedding is a tunable parameter

Experimenting with different embedding dimensions helps determine the most effective representation of words in the context of a particular task.

Embedding layer

	Dim1	Dim2	Dim3	Dim4	Dim5	Dim6	Dim7	Dim8
king	0.339316	0.780608	0.244805	0.271532	0.930791	0.991278	0.178408	0.916366
queen	0.416181	0.430735	0.527269	0.803323	0.489871	0.512701	0.535537	0.013570
man	0.691552	0.688406	0.638646	0.592562	0.194003	0.723072	0.223624	0.576644
woman	0.367290	0.591545	0.383847	0.445769	0.418985	0.026878	0.163643	0.037023
apple	0.779587	0.380875	0.651247	0.988765	0.780108	0.166571	0.282808	0.226567
orange	0.976849	0.326953	0.339139	0.822603	0.331933	0.466903	0.587653	0.366683
car	0.881887	0.040636	0.275993	0.530267	0.368584	0.333479	0.003362	0.869546
bicycle	0.809213	0.097499	0.922652	0.392323	0.900413	0.670873	0.851242	0.736461

the weights associated with the embeddings are initialized randomly

weights are iteratively adjusted using backpropagation

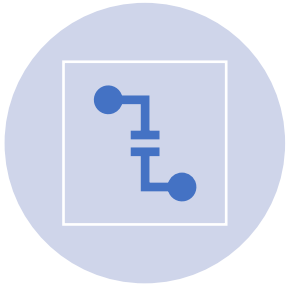
Ultimately, the trained Embedding layer produces embeddings that encode nuanced similarities between words.

Hidden Layer

This one-hot encoded vector is then multiplied by an input weight matrix W (of size V x N, where N is the desired size of word embeddings) to produce the hidden layer.

Given that the input is one-hot encoded, this multiplication essentially picks out the row corresponding to the input word.

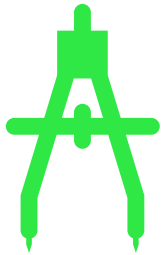
If multiple context words are used (as in our case), their resulting vectors are averaged to produce a single hidden layer representation.



The hidden layer doesn't have an activation function in the traditional Word2Vec architecture. The representation in this layer is essentially our word vector.

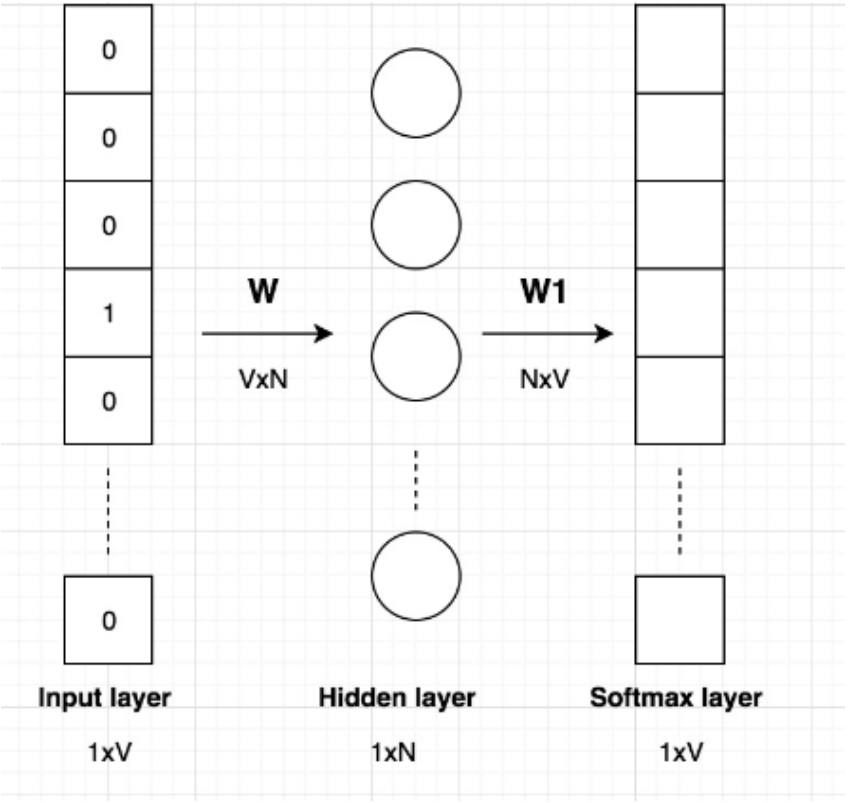
Output & Softmax

The hidden layer representation is then multiplied by an output weight matrix W' (of size N x V).



This produces a vector of size V, where each entry corresponds to a score for each word in the vocabulary.

To turn these scores into probabilities, a softmax function is applied. The softmax function ensures that the sum of all word probabilities is 1.

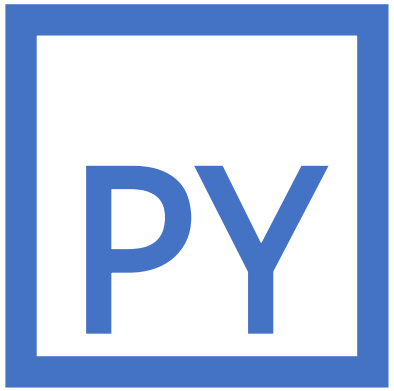


$$[0\,0\,0\,0\,1\,0] \times \begin{bmatrix} 10 & 23 & 15 \\ 3 & 14 & 9 \\ 18 & 26 & 2 \\ 10 & 17 & 7 \\ 12 & 23 & 8 \\ 9 & 10 & 12 \end{bmatrix} = [12\ \ 23\ \ 8]$$

Extraction of Word Vector:

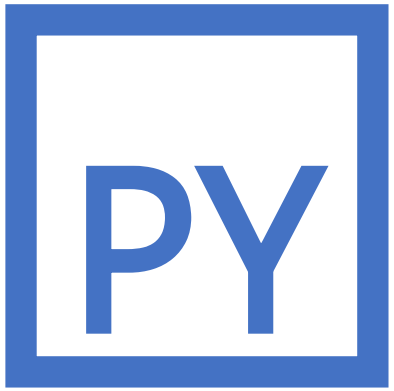
- After training, the word vectors (or embeddings) for words can be extracted from the input weight matrix W.
- Specifically, the row corresponding to each word in this matrix represents its embedding.
- For example, to get the vector for the word "says", you would look at the 5th row (from our previous example) of matrix W.

Demo using python/sklearn



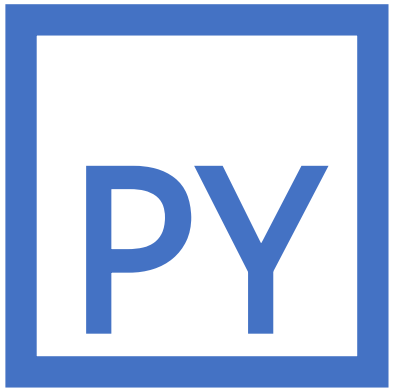
Use of embedding layer (Keras/TF – IMDB classification

Demo using python/sklearn



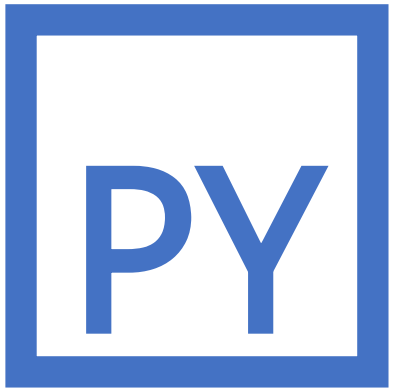
simple word2vec using dummy data

Demo using python/sklearn



using GENSIM for training word2vec

Demo using python/sklearn



Example use cases with GENSIM

Thanks!